Valve Inventory

Technical Manual

Architecture, schema, and internals - for anyone extending the tool, scripting against the database directly, or just wanting to know how a feature actually works under the hood.

1. Architecture

One SQLite database, two independent front ends that read and write it directly - there is no server, no API layer, and no ORM. Both front ends import `valvelib.py` for the schema and shared logic.

File	Role
valvelib.py	Schema (SCHEMA string, executed via executescript), type-name normalisation (norm()), and the naming-convention classifier (classify()).
valves.py	Command-line front end. One cmd_* function per subcommand, dispatched via argparse.
valves_gui.py	Tkinter desktop front end. A single App(ttk.Frame) class holding four tabs' worth of widgets and handlers.
build_db.py	One-off converter from the original 38-tab spreadsheet. Already run; kept for provenance, not part of normal operation.
snapshot.py	Writes/restores the data/ text snapshot that stands in for the binary database in version control.
fetch_datasheets.py	Two-stage, rate-limited crawler/downloader for the local datasheet archive (frank.pocnet.net only).
import_researched.py	Parses a Claude research reply (block format below) and applies it to valve_type.
test_smoke.py	No framework - a flat script of check()/check_true() calls, exits non-zero on first failure.

2. Database schema

Five tables plus one convenience view, declared in `valvelib.SCHEMA` and created with `CREATE TABLE IF NOT EXISTS` - so adding a table to the schema and re-running `V.init_db()` (which both front ends do on every startup) is enough to migrate an existing database with no separate migration step.

valve_type - one row per type, the reference library

Primary key `type_key` (normalised: uppercase, alphanumeric only - see norm() below). Columns: name, function, family, base, pins, heater_v, heater_a, va_max, pa_max, gm, mu, power_out, freq_max, typical_use, equivalents (space-separated), datasheet_path, datasheet_url, confidence (`inferred` | `confirmed`), notes.

stock - one row per physical lot

type_key (FK to valve_type, ON UPDATE CASCADE), box, qty, manufacturer, condition, date_added, notes. The box identifier is free text, not an enforced foreign-key relationship - the separate box table below just carries optional per-box location/label notes keyed on that same identifier.

socket - one row per lot of bases/sockets

base, box, qty, condition, notes. Split out from sundry deliberately, so a base type is a first-class, searchable thing rather than free text.

sundry / box

sundry is the general catch-all for non-valve, non-socket items (crystals, screening cans, chimneys). box holds per-box location/label notes, keyed on the same free-text box identifier used in stock and socket.

v_stock (view)

stock LEFT JOIN valve_type, exposing the commonly-needed combined columns (type name, function, heater_v, pa_max, freq_max, base, datasheet_path) without repeating the join in every query.

> **Note:** The type_key -> stock.type_key foreign key is declared `ON UPDATE CASCADE`. That matters in practice: renaming a type_key (e.g. correcting a mis-entered designation) is a single UPDATE on valve_type, done through a connection with foreign_keys=ON (V.connect() sets this) - stock rows follow automatically, no manual UPDATE stock needed.

3. Type-name normalisation and classification

norm(name)

Uppercases, strips a leading JAN/JAN-/CV- service prefix, then strips everything that isn't A-Z0-9. “jan 7289” → “7289”; “PY 4-400” → “PY4400”. This is the type_key used everywhere - display names keep their original punctuation in `name`.

classify(name)

Infers function/heater/family from the designation alone, trying each national naming convention in turn until one matches: a curated KNOWN table (hand-entered for designations no rule covers) first, then GEC beam tetrodes (KT-prefix), European Mullard/Philips codes, British Mazda/Brimar codes, Russian codes, American RETMA codes, British GEC/Osram codes, then transmitting-type patterns (4CX/3-500Z-style). Returns whatever it's confident about and leaves the rest blank - see the code comments for exactly which patterns are accepted and why (several national schemes collide on the same letter/number patterns, so order and specific guard conditions matter).

> **Note:** classify() output is always confidence='inferred'. Nothing sets confidence to confirmed except a human (Save + confirm in the GUI, --confirm on the CLI) or import_researched.py applying a research result that wasn't hedged.

4. GUI internals worth knowing

Equivalents-aware search

run_search() builds the normal filtered query, then add_equivalent_rows() checks whether the Text field names an exact type_key; if so it pulls in stock for every type_key that either appears in that type's own equivalents field, or has this type's key in *its* equivalents field (checked both directions in Python, not SQL, since equivalents is unstructured free text). Matched rows get a `match` label and the `equiv` tree tag.

Similar-types suggestions

find_similar() buckets the current type's function text via function_group() (a simple first-match keyword scan against valvelib.FUNCTION_GROUPS), then scans every other held type in the same bucket, requiring every field both types have a value for (va_max, pa_max, gm, mu, power_out, freq_max) to be within SIMILAR_TOLERANCE (50%) of the reference. Heater is deliberately excluded from the match test and checked separately as a flag - a different heater rating doesn't rule out a substitute, it just needs a note.

Browse tab - Category vs. Function

The raw function text is specific enough that almost every type has a unique value, which makes it useless as a browse facet. browse_category() maps it to a coarser, purpose-built bucket list (BROWSE_CATEGORIES) instead - compound types are checked before the simpler categories they'd otherwise match as a substring (“triode-pentode” before “triode”). variable_mu is a second derived, orthogonal flag (checks for “variable-mu” / “remote cutoff” wording), since that property cuts across pentode and tetrode rather than being its own category.

Because category/variable_mu aren't real columns, the whole Browse tab filters in Python over the full (small, ~250-row) type list rather than building SQL per field - see pb_load_all() / pb_matches() / pb_refresh_dropdowns(). At this scale that's faster to reason about than parallel SQL and Python category logic, and cascading dropdown options (pb_matches(..., exclude=field)) fall out of the same predicate function for free.

Repair Bench - composition over duplication

Nothing about identifying an unknown valve or proposing a substitute is new logic - the tab (rb_* methods) is built entirely from pieces the other tabs already needed: find_similar() for substitute candidates (reused as-is, then filtered to held_qty > 0 - a substitute you don't have isn't useful mid-repair), do_open_sheet(row) / do_lookup(site, row) for datasheet and web lookups (both already took an optional explicit row dict rather than always reading the Valves tab's selection, precisely so other tabs could call them), and the same bidirectional equivalents scan used by the Valves tab's search (factored out here as rb_find_matches() rather than shared directly, since the Valves tab version is itself embedded in a larger SQL query builder). save_type() was split into apply_type_fields(key, field_vars, notes_widget, confirm) plus a thin wrapper specifically so Repair Bench's Save button could reuse the exact same validate-and-write logic against its own, separate form widgets.

5. The research/import pipeline

Both the electrical-parameters prompt and the datasheet-download prompt are generated from the live gap list at the moment you ask for them (Tools menu), not hard-coded - re-running them later reflects whatever's still unconfirmed then.

Expected reply format

```
TYPE_NAME
function:
base:
pins:
heater_v:
heater_a:
va_max:
pa_max:
gm:
mu:
power_out:
freq_max:
typical_use:
equivalents:
source_url:
confidence_note:
---
```

import_researched.py's parse_blocks() splits on the `---` separator, normalises the first line of each block through the same norm() used everywhere else, and pulls a bare number out of each numeric field with a permissive regex (so “250 V” or “~250” still parses). A block whose only non-blank field is equivalents is skipped entirely, on the basis that “we already knew that” isn't a research result worth recording.

Confirmed vs. lead-only

apply_records() checks confidence_note against a fixed list of hedge phrases (HEDGE_WORDS: “could not”, “low confidence”, “plausible”, “unconfirmed”, and similar). If any match, the fields are still written - the data is a genuine lead worth keeping - but confidence stays `inferred` rather than flipping to `confirmed`. This is the same policy applied by hand throughout this collection's own research pass; see the notes field on any type for a worked example.

6. CLI command reference

Command	Purpose
box BOX	List a box's contents.
find TYPE	Which boxes hold a type (follows equivalents).
show TYPE	Full reference record for a type.
search --function .. --heater .. --pa ..	Parametric search; numeric flags take a bare value or a >, <, >=, <= comparison.

Command	Purpose
add TYPE --box N [--qty --maker --condition --notes]	Add stock; creates the type automatically if new.
import-csv FILE	Bulk-add stock from a CSV (see upload_template.csv).
take TYPE [--qty --box]	Remove stock you've used.
move TYPE --frm A --to B	Move a type between boxes.
bases [--base --box]	List valve base/socket stock.
sock-add / sock-take / sock-move	Same idea as add/take/move, for bases/sockets.
set TYPE --pa .. --va .. --confirm	Edit a type's reference parameters; --confirm marks it confirmed.
merge SRC DST [--yes]	Fold one type into another - moves stock, records SRC as an equivalent of DST. Dry-run without --yes.
dupes	Candidate duplicate type pairs, for review before merge.
scan [--archive]	Link local datasheet files into the database by filename.
sheet TYPE [--open]	Locate (and optionally open) a type's local datasheet.
gaps [--limit]	What still needs data, ordered by quantity held.
stats	Collection summary.
export [path]	Write an .xlsx snapshot (requires openpyxl).

7. Version control and the snapshot design

`valves.db` is gitignored. SQLite databases are binary blobs - git can store them but every single-row edit rewrites the whole file, so diffs are meaningless and history is bloated. snapshot.py writes the same content as plain CSV (one file per table) plus a full SQL dump (`valves.sql`, via `con.iterdump()`), all under `data/`. That's what's actually committed - readable diffs for every change to the collection, and a restore path that doesn't depend on the SQLite file format being stable across versions.

> **Note:** iterdump() emits tables in alphabetical order, which lists stock rows before valve_type exists. restore() explicitly runs with foreign_keys=OFF while loading the dump for exactly this reason, then reports (rather than silently ignoring) any row that fails PRAGMA foreign_key_check afterward.

8. Privacy note on distributing this data

The typical_use and notes fields carry descriptive text originally gathered from r-type.org during the original conversion - not something to republish freely. snapshot.py --strip-notes (and the equivalent checkbox in File > Export archive and tools) omits those two fields from the export while leaving every classification, parameter, and box location intact. Datasheet PDFs are gitignored and never included in an export for the same reason - they're third-party files, meant to be rebuilt locally by whoever needs them.

9. Extending the tool

- **New reference field** - add the column to SCHEMA in valvelib.py, then to TYPE_FIELDS in valves_gui.py (drives the detail-panel form) and ALL_FIELDS in import_researched.py if research prompts should be able to fill it. No migration step needed - CREATE TABLE IF NOT EXISTS plus ALTER TABLE-free additive columns would need a small ALTER TABLE ADD COLUMN in init_db() if adding to an *existing* table; the tables here were designed with the fields we currently use, so extending one is the one schema change that isn't fully automatic.
- **New CLI command** - one cmd_*(con, a) function in valves.py plus an add_parser() block; follow an existing command for the pattern.
- **New GUI tab** - add a _build_x_tab(root) method, call it from App.__init__ alongside the existing four, and give it its own prefixed method names (px_*) to avoid colliding with the other tabs' state.